Deliveroojs APIs

Lab 1: Introduction to Agent Development

Building autonomous agents for the Deliveroo.js environment

Learning Objectives

In this lab, we will learn how to interact with the Deliveroojs APIs, implementing a script that acts as a simple agent:

- **Navigate** along a predefined path
- **Pick up** parcels from the environment
- **Deliver** parcels to their destinations

Deliveroo.js

Deliveroo.js it's a **Grid-Based** parcel collection videogame. You can play in **Multi-Player** from your browser or develop your own Autonomous Agent that plays in your place.

Where to play

Play on the web at https://deliveroojs.azurewebsites.net/, or preview the latest `develop` version at https://deliveroojsdev.azurewebsites.net/

Within unitn network at https://deliveroojs.bears.disi.unitn.it/ (Requires unitn VPN which can be installed following instructions available at https://servicedesk.unitn.it/sd/it/service/vpn?id=unitrento_v2_service_card&sys_id=e5a898fec35bfd104cbb7055df013171).

To run locally:

1. `git clone https://github.com/unitn-ASA/Deliveroo.js.git`
2. `cd Deliveroo.js`, `npm run build`, `npm start`

How to play

```
As user:       `a`: left, `s`: down, `d`: right, `w`: up,
               `q`: pickup, `e`: putdown
As an admin:   press `space` to create or dispose parcels
               and numeric keys to modify tiles.
```

To start developing your own scripts

- Create an empty npm project `npm init`
- Install the SDK npm package `npm install @unitn-asa/deliveroo-js-sdk`
- Import utilities classes and types from the sdk package

```
// See https://github.com/unitn-ASA/Deliveroo.js/tree/master/packages
import { DjsConnect, DjsClientSocket } from "@unitn-asa/deliveroo-js-sdk/client";
```

- Examples are available at github.com/unitn-ASA/DeliverooAgent.js.

Deliveroo.js SDK Architecture

The SDK provides three main client classes:

DjsClientSocket

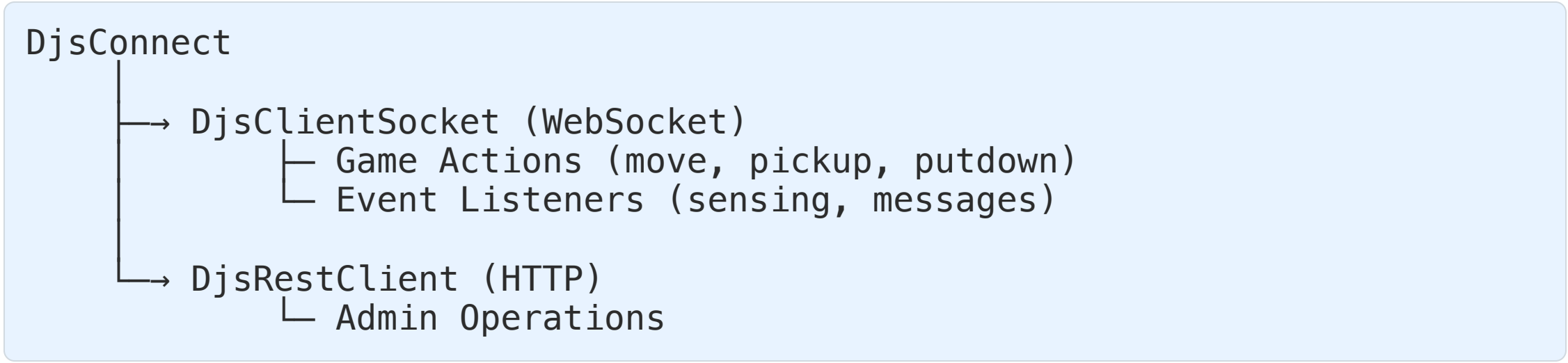
Real-time Socket.io client for game actions and events

DjsRestClient

HTTP RESTful API for CRUD operations

DjsConnect

Factory class for authentication and connection


DjsConnect
DjsClientSocket (WebSocket)
Game Actions (move, pickup, putdown)
Event Listeners (sensing, messages)
DjsRestClient (HTTP)
Admin Operations

Connection & Authentication

Method 1: Hardcoded

```
const socket = DjsConnect( 'localhost:8080', 'YOUR_TOKEN' } );
```

Method 2: Environment Variables

```
# This is file .env
HOST=localhost:8080
TOKEN=your_token_here
# or
NAME=MyAgentName
```

Key Game Actions

All actions return **Promises** for async/await usage:

Movement

```
const result = await socket.emitMove('up'); // 'right', 'left', 'down'
// Returns: {x, y} on success, false on failure
```

Pickup Parcels

```
const parcels = await socket.emitPickup();
// Returns: [{id, x, y, carriedBy, reward}, ...]
```

Drop Parcels

```
const dropped = await socket.emitPutdown(selected);
// selected: array of parcel IDs (or undefined for all)
```

Event Listeners

The API uses Node.js **EventEmitter** pattern:

```
// Agent identity
socket.on('you', (id, name, x, y, score) => { /* ... */ });

// Map information
socket.on('map', (width, height, tiles) => { /* ... */ });
socket.on('tile', (x, y, delivery) => { /* ... */ });

// Sensing
socket.on('agentsSensing', (agents) => { /* ... */ });
socket.on('parcelsSensing', (parcels) => { /* ... */ });

// Connection
socket.on('connect', () => { /* ... */ });
socket.on('disconnect', () => { /* ... */ });
```

Data Structures

```
IOAgent: {
    id: string,
    name: string,
    teamId: string,
    x: number, y: number,
    score: number,
    penalty: number
}
IOParcel: {
    id: string,
    x: number, y: number,
    carriedBy?: string,
    reward: number
}
IOTile: {
    x: number, y: number,
    type: '0'|'1'|'2'|'3'|'4'|'5'|'←'|'↑'|'→'|'↓'
    // 0: wall, 1: spawner, 2: delivery, 3: walkable, 4: base
}
```

Complete Agent Example

```
import { DjsConnect } from '@unitn-asa/deliveroo-js-sdk';

const socket = DjsConnect('localhost:8080');

let myPosition = { x: 0, y: 0 };

socket.on('you', (id, name, x, y) => {
    myPosition = { x, y };
});

socket.on('map', async (width, height, tiles) => {
    // Move along predefined path
    const path = ['right', 'right', 'down', 'down', 'left', 'left', 'up', 'up'];

    for (const direction of path) {
        const result = await socket.emitMove(direction);
        if (!result) {
            console.log(`Move ${direction} failed, retrying...`);
            await new Promise(r => setTimeout(r, 100));
            // Retry logic here
        }
    }

    // Pickup parcels
    await socket.emitPickup();
});
```

REST API Endpoints

Via `DjsRestClient` or direct HTTP calls:

Method	Endpoint	Purpose
GET	`/api/agents`	List all agents
GET	`/api/agents/:id`	Get specific agent
GET	`/api/parcels`	List all parcels
POST	`/api/tokens`	Generate auth token
POST	`/api/agents`	Create new agent
PATCH	`/api/configs`	Update configuration
DELETE	`/api/parcels/:id`	Remove parcel

Check API documentation at https://deliveroojs.bears.disi.unitn.it/api-docs/

Example: Handling Failures

When multiple agents compete for resources:

```
async function resilientMove(direction, maxRetries = 3) {
    for (let i = 0; i < maxRetries; i++) {
        const result = await socket.emitMove(direction);
        if (result) return result;

        // Wait before retry
        await new Promise(resolve => setTimeout(resolve, 500));
    }

    // Alternative path or request help
    await socket.emitShout(`Help! Blocked trying to move ${direction}`);
    return null;
}
```

Summary

Key Concepts

- **Promise-based actions** for all game operations
- **Event-driven architecture** for real-time updates
- **Flexible authentication** via token or name
- **Built-in communication** between agents

Implement an agent that:

- Moves along a predefined path
- Pick the parcel
- Deliver it

What if other agents are moving?

- Dealing with failing actions, by insisting on path.

Next Steps

- Implement basic navigation